

1.0 大模型部署

1.1 LLM 部署方式



1.1.1 本地部署

用 ollama(大模型运行工具)在自己电脑上下载 LLM 进行对话(由于电脑配置不好,暂不部署)

1.1.2&1.1.3 官方 api&第三方平台

在官方平台注册,充值,建 apik;

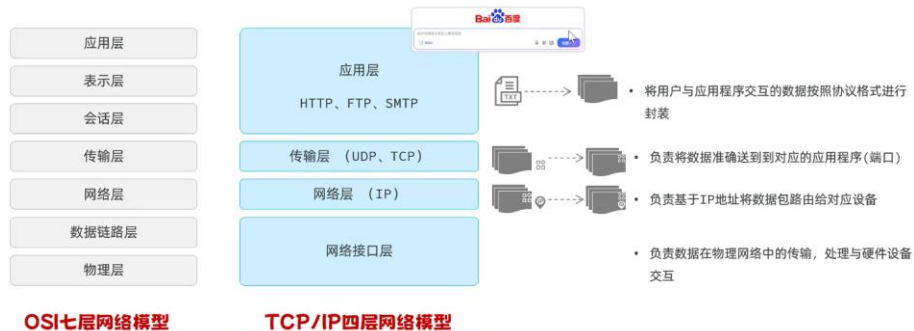
1.2 LLM 调用基础知识



网络模型

互联网 (Internet) 连接了数以亿计的设备, 网络四通八达就如同时城市的复杂的道路。网络中传输的数据就像是城市道路中的车流, 如果不加以管理那必然会出现混乱。在国际ISO组织就统一了程序在**网络中通信的模型**和标准。包括:

- **OSI网络模型**: 全球网络互连标准模型
- **TCP/IP网络模型**: 可以认为是OSI的简化版



计网简单复习;

HTTP协议

- 概念: **Hyper Text Transfer Protocol**, 超文本传输协议, 规定了客户端和服务端之间数据传输的规则。(只有在请求及响应中都遵循了统一的规则, 服务端才能读懂客户端发送来的请求, 客户端才能解析服务端响应的结果)



- 特点:
 1. 基于文本的协议: 请求和响应的部分的协议内容为文本格式, 底层通过TCP协议传输, 稳定性强。
 2. 基于请求-响应模型: 一次请求对应一次响应, 必须由客户端先发起请求, 服务端才会返回响应。
 3. 无状态: 服务端不会记忆与客户端的历史交互信息, 每次请求-响应都是独立的。

HTTP协议-请求数据格式

```
POST /api/courses HTTP/1.1
Accept: application/json, text/plain, */*
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
Connection: keep-alive
Content-Length: 139
Content-Type: application/json
Host: localhost:90
Origin: http://localhost:90
Referer: http://localhost:90/resource/course
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/143.0.0.0

{"phone":"18808088080","channel":1,"name":"虎咆","gender":1,"age":32}
```

请求行(请求方式、资源路径、协议)

行,头,体;

请求头(格式key: value)

请求体(请求参数部分, GET方式没有, POST可以有)

请求方式:

- GET: 请求参数在请求行中, 没有请求体。如: `/api/courses?name=Python&status=1`。GET请求请求参数大小在浏览器中是有限制的。
- POST: 请求参数在请求体中, POST请求大小是有限制的。

get 只读;post 可修改;

HTTP协议-响应数据格式

HTTP/1.1 200 Server: nginx/1.24.0 Date: Tue, 16 Dec 2025 12:38:08 GMT Content-Type: application/json Transfer-Encoding: chunked Connection: keep-alive	响应行(协议、状态码) 响应头(格式key: value)
{code: 1, msg: "success", data: null}	响应体(存放服务端响应的数据)

200: 客户端请求成功

400: 请求参数错误

404: 请求资源不存在, url输入有误, 或者网站资源被删除了

500: 服务器发生了不可预期的错误

响应体一般为 json 格式;注意请求/响应头 和 体之间的空行;

1.3 LLM 调用:apifox(如 postman)测试

由于浏览器获取数据都是 get 请求,获得的信息有限;而与 LLM 的信息交换可能含有大量提示词(nk),故调用 LLM 用 post 请求,不是通过浏览器的方式;

Deepseek 调用文档:

调用对话 API

在创建 API key 之后, 你可以使用以下样例脚本的来访问 DeepSeek API。样例为非流式输出, 您可以将 stream 设置为 true 来使用流式输出。

curl python nodejs

url地址

请求头(key: value)

请求体(json格式)

```
curl https://api.deepseek.com/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer ${DEEPSEEK_API_KEY}" \
-d '{
  "model": "deepseek-chat",
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Hello!"}
  ],
  "stream": false
}'
```

url 地址,请求头放 headers,请求体放 Body;

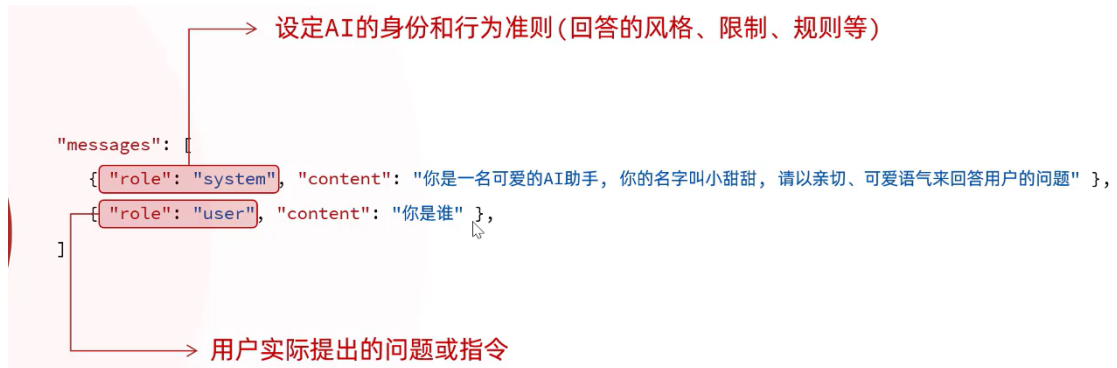
Json格式

JSON(JavaScript Object Notation)是前端的一种对象表示方法。表示形式类似于Python中的字典,都是 key:value这种形式,不过所有的key都必须使用双引号引起来,值可以是任何类型:

- **对象**: 用{}表示, {}之间是键值对形式, 键是字符串, 值可以是任意其它类型
- **数字**: 整数和小数都是数字, 例如: 12、3.14
- **字符串**: 用""引起来, 例如: "jack"
- **布尔**: 有两种值: true或false
- **列表**: 用[]表示, []中是列表的元素, 多个元素以,分割。

```
{
  "model": "deepseek-chat",
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Hello!"}
  ],
  "stream": false
}
```

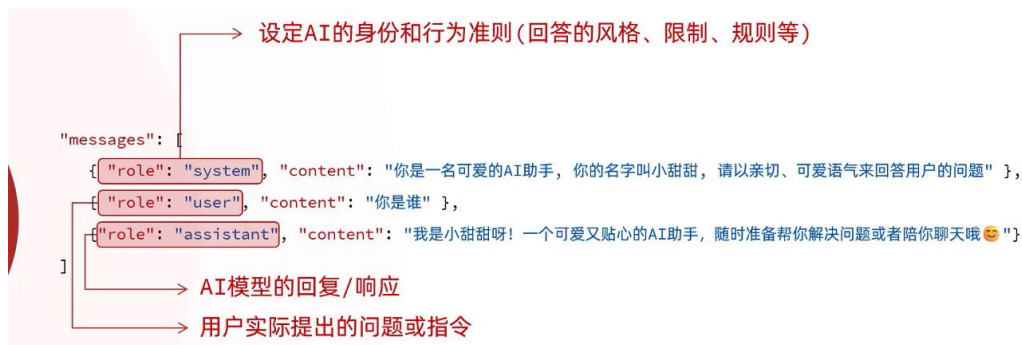
与 py 有些相似;



1.4 LLM 的对话记忆

由于 Http/s 请求是无状态的,LLM 并没有与我们对话的上下文,导致对话不连贯;

解决: 在请求体中,加上之前的问答记录(json 对象格式)



1.5 py 代码调用 LLM



Py 中包下载;

- 1>在环境变量中设置 DEEPSEEK_API_KEY 即 apik 的值;(此电脑-属性-高级设置-环境变量)
- 2>代码调用;

```
# Please install OpenAI SDK first: `pip3 install openai`

import os#py中os的包
from openai import OpenAI#openai的调用包

# 创建与LLM交互的客户端对象(DEEPSEEK_API_KEY为环境变量名字, 值为apik)
client = OpenAI(
    api_key=os.environ.get('DEEPSEEK_API_KEY'),#从本机环境变量中获取apik, 首次设置环境变量后, 要重启ide
    base_url="https://api.deepseek.com")
#交互参数设置
response = client.chat.completions.create(
    model="deepseek-v4-pro",
    messages=[
        {"role": "system", "content": "You are a helpful assistant"},
        {"role": "user", "content": "Hello"},
    ],
    stream=False,
    reasoning_effort="high",
    extra_body={"thinking": {"type": "enabled"}}
)

print(response.choices[0].message.content)#打印返回信息,response即为返回的json响应体, 调用下层用。
```

1.6 prompt 工程



- 提示词 (Prompt): 是引导大模型 (LLM) 进行内容生成的命令 (一句话、一个问题等)。
- 提示词工程 (Prompt Engineering): 通过有技巧的编写提示词, 使大模型生成出尽可能符合预期的内容, 这一持续性的过程就称为提示词工程。

